

3.3 Boolean Expressions

An `if` statement like

```
if (x > max)
    max = x;
```

is perhaps deceptively simple. Conceptually, we say that it checks whether the condition `x > max` is ``true" or ``false". The mechanics underlying C's conception of ``true" and ``false," however, deserve some explanation. We need to understand how true and false values are represented, and how they are interpreted by statements like `if`.

As far as C is concerned, a true/false condition can be represented as an integer. (An integer can represent many values; here we care about only two values: ``true" and ``false." The study of mathematics involving only two values is called Boolean algebra, after George Boole, a mathematician who refined this study.) In C, ``false" is represented by a value of 0 (zero), and ``true" is represented by any value that is nonzero. Since there are many nonzero values (at least 65,534, for values of type `int`), when we have to pick a specific value for ``true," we'll pick 1.

The *relational operators* such as `<`, `<=`, `>`, and `>=` are in fact operators, just like `+`, `-`, `*`, and `/`. The relational operators take two values, look at them, and ``return" a value of 1 or 0 depending on whether the tested relation was true or false. The complete set of relational operators in C is:

<code><</code>	less than
<code><=</code>	less than or equal
<code>></code>	greater than
<code>>=</code>	greater than or equal
<code>==</code>	equal
<code>!=</code>	not equal

For example, `1 < 2` is 1, `3 > 4` is 0, `5 == 5` is 1, and `6 != 6` is 0.

We've now encountered perhaps the most easy-to-stumble-on ``gotcha!" in C: the equality-testing operator is `==`, not a single `=`, which is assignment. If you accidentally write

```
if (a = 0)
```

(and you probably will at some point; everybody makes this mistake), it will *not* test whether `a` is zero, as you probably intended. Instead, it will *assign* 0 to `a`, and then perform the ``true" branch of the `if` statement if `a` is *nonzero*. But `a` will have just been assigned the value 0, so the ``true" branch will never be taken! (This could drive you crazy while debugging--you wanted to do something if `a` was 0, and after the test, `a` is 0, whether it was supposed to be or not, but the ``true" branch is nevertheless not taken.)

The relational operators work with arbitrary numbers and generate true/false values. You can also combine true/false values by using the *Boolean operators*, which take true/false values as operands and compute new true/false values. The three Boolean operators are:

<code>&&</code>	and
<code> </code>	or
<code>!</code>	not (takes one operand; ``unary'')

The `&&` ("and") operator takes two true/false values and produces a true (1) result if both operands are true (that is, if the left-hand side is true **and** the right-hand side is true). The `||` ("or") operator takes two true/false values and produces a true (1) result if either operand is true. The `!` ("not") operator takes a single true/false value and negates it, turning false to true and true to false (0 to 1 and nonzero to 0).

For example, to test whether the variable `i` lies between 1 and 10, you might use

```
if (1 < i && i < 10)
    ...
```

Here we're expressing the relation "`i` is between 1 and 10" as "`1` is less than `i` **and** `i` is less than 10."

It's important to understand why the more obvious expression

```
if (1 < i < 10)                /* WRONG */
```

would not work. The expression `1 < i < 10` is parsed by the compiler analogously to `1 + i + 10`. The expression `1 + i + 10` is parsed as `(1 + i) + 10` and means "add 1 to `i`, and then add the result to 10." Similarly, the expression `1 < i < 10` is parsed as `(1 < i) < 10` and means "see if 1 is less than `i`, and then see if the result is less than 10." But in this case, "the result" is 1 or 0, depending on whether `i` is greater than 1. Since both 0 and 1 are less than 10, the expression `1 < i < 10` would *always* be true in C, regardless of the value of `i`!

Relational and Boolean expressions are usually used in contexts such as an `if` statement, where something is to be done or not done depending on some condition. In these cases what's actually checked is whether the expression representing the condition has a zero or nonzero value. As long as the expression is a relational or Boolean expression, the interpretation is just what we want. For example, when we wrote

```
if (x > max)
```

the `>` operator produced a 1 if `x` was greater than `max`, and a 0 otherwise. The `if` statement interprets 0 as false and 1 (or any nonzero value) as true.

But what if the expression is not a relational or Boolean expression? As far as C is concerned, the controlling expression (of conditional statements like `if`) can in fact be *any* expression: it doesn't have to "look like" a Boolean expression; it doesn't have to contain relational or logical operators. All C looks at (when it's evaluating an `if` statement, or anywhere else where it needs a true/false value) is whether the expression evaluates to 0 or nonzero. For example, if you have a variable `x`, and you want to do something if `x` is nonzero, it's possible to write

```
if (x)
    statement
```

and the statement will be executed if `x` is nonzero (since nonzero means "true").

This possibility (that the controlling expression of an `if` statement doesn't have to "look like" a Boolean expression) is both useful and potentially confusing. It's useful when you have a variable or a function that is "conceptually Boolean," that is, one that you consider to hold a true or false (actually nonzero or zero) value. For example, if you have a variable `verbose` which contains a nonzero value when your program should run in verbose mode and zero when it should be quiet, you

can write things like

```
if(verbose)
    printf("Starting first pass\n");
```

and this code is both legal and readable, besides which it does what you want. The standard library contains a function `isupper()` which tests whether a character is an upper-case letter, so if `c` is a character, you might write

```
if(isupper(c))
    ...
```

Both of these examples (`verbose` and `isupper()`) are useful and readable.

However, you will eventually come across code like

```
if(n)
    average = sum / n;
```

where `n` is just a number. Here, the programmer wants to compute the average only if `n` is nonzero (otherwise, of course, the code would divide by 0), and the code works, because, in the context of the `if` statement, the trivial expression `n` is (as always) interpreted as ```true``` if it is nonzero, and ```false``` if it is zero.

``Coding shortcuts" like these can seem cryptic, but they're also quite common, so you'll need to be able to recognize them even if you don't choose to write them in your own code. Whenever you see code like

```
if(x)
```

or

```
if(f())
```

where `x` or `f()` do not have obvious ```Boolean``` names, you can read them as ```if x is nonzero``` or ```if f() returns nonzero```.

Read sequentially: [prev](#) [next](#) [up](#) [top](#)

This page by [Steve Summit](#) // [Copyright](#) 1995, 1996 // [mail feedback](#)